



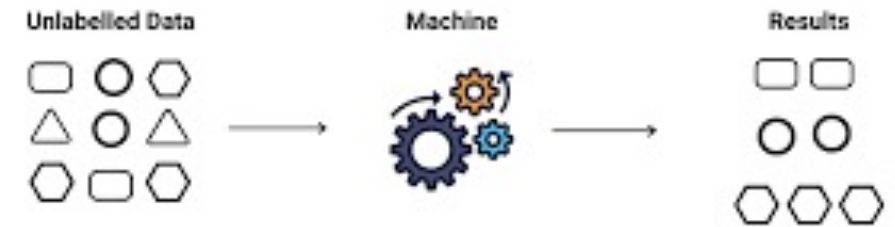
Unsupervised ML

Dr. Ahmed Aljarah

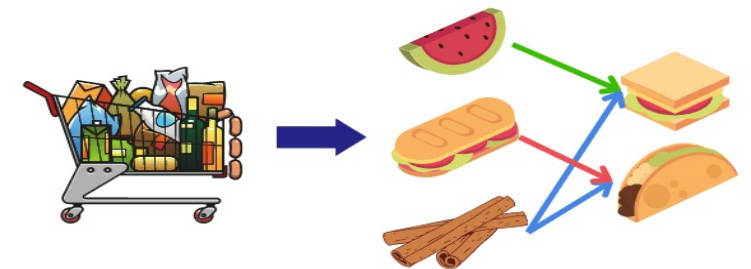
Unsupervised ML

- The model learns from data without labels.
- There are no correct answers given, so the model looks for patterns, structures, or groups by itself.
 - Clustering: grouping similar data points.
 - Association: discovering hidden patterns or relationships in data.

Unsupervised Learning



Clustering



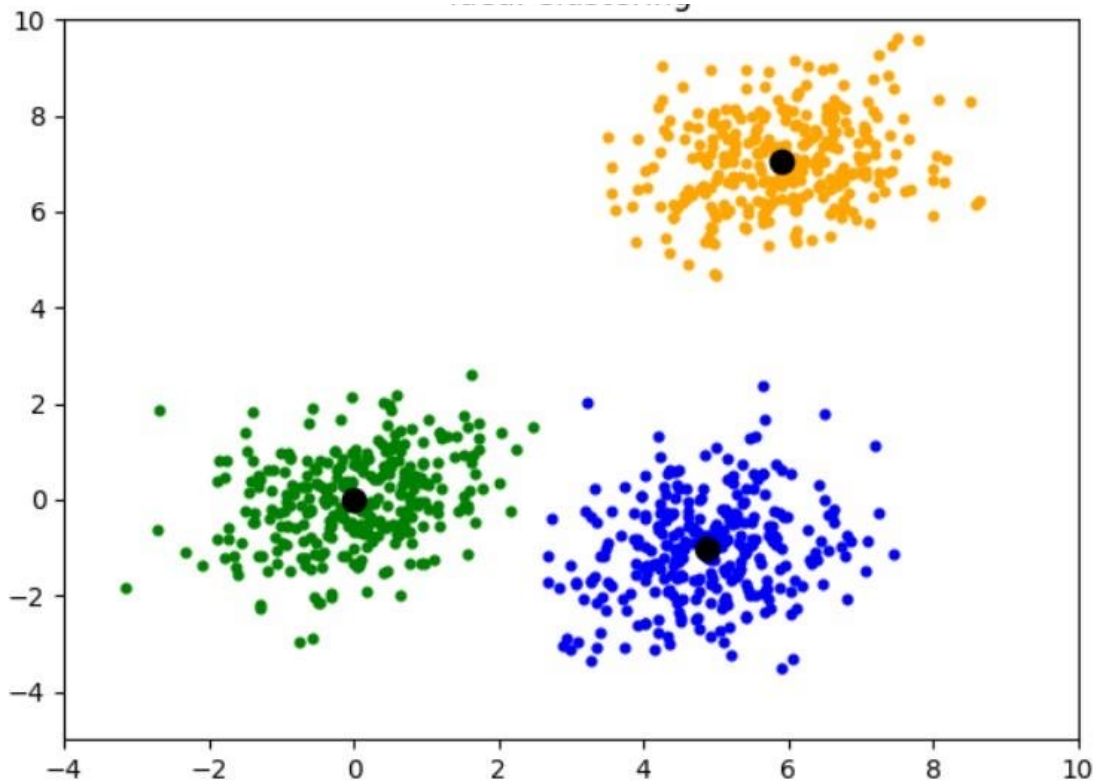
Association

Unsupervised applications

- Market segmentation
Customers grouped by age, income, or buying habits
- Social network analysis
Studies relationships and connections between people
- Organization of computational clusters
Google uses computational clusters to process huge amounts of data efficiently.
- Market Basket Analysis
Used by supermarkets and e-commerce companies to analyze customer purchases.
- Astronomical data analysis
Detecting new planets using data from space telescopes.
- Big Data
Social media data, online transactions, and sensor data.



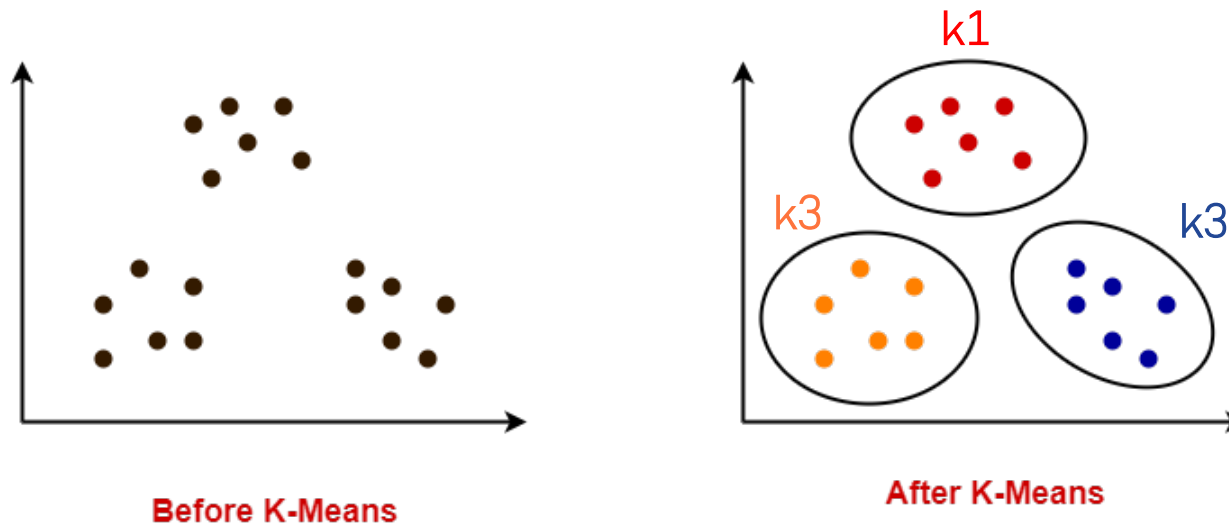
Clustering



- Clustering is a technique used to group similar data points together.
- K-Means is one of the most widely used for clustering algorithms.

K-Means algorithm

- K-Means Clustering is a distance-based unsupervised learning algorithm that groups similar data points into clusters without requiring labeled data.
- Its main purpose is to discover hidden patterns and organize data according to similarity.
- The algorithm works by assigning points to the nearest cluster center based on distance measurements.



K-Means algorithm steps

1. Choose the number of clusters (K).
2. Initialize K centroids randomly or using K-Means++.
3. Assign each data point to the nearest centroid.
4. Recalculate the centroids by computing the mean of each cluster.

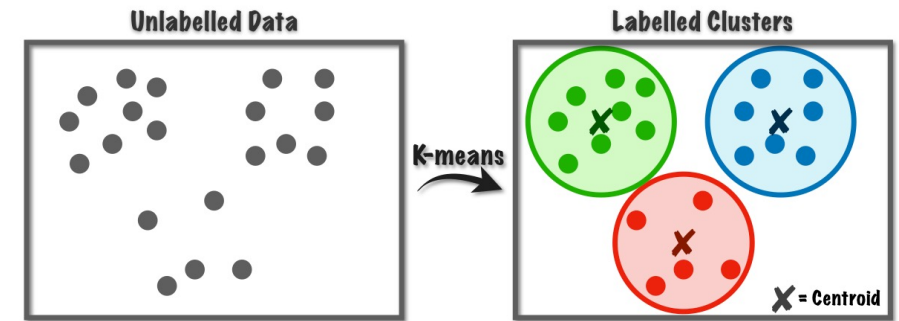
$$\mu_k = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{where}$$

μ_k = centroid of cluster k

n = number of points in the cluster

x_i = data points in the cluster

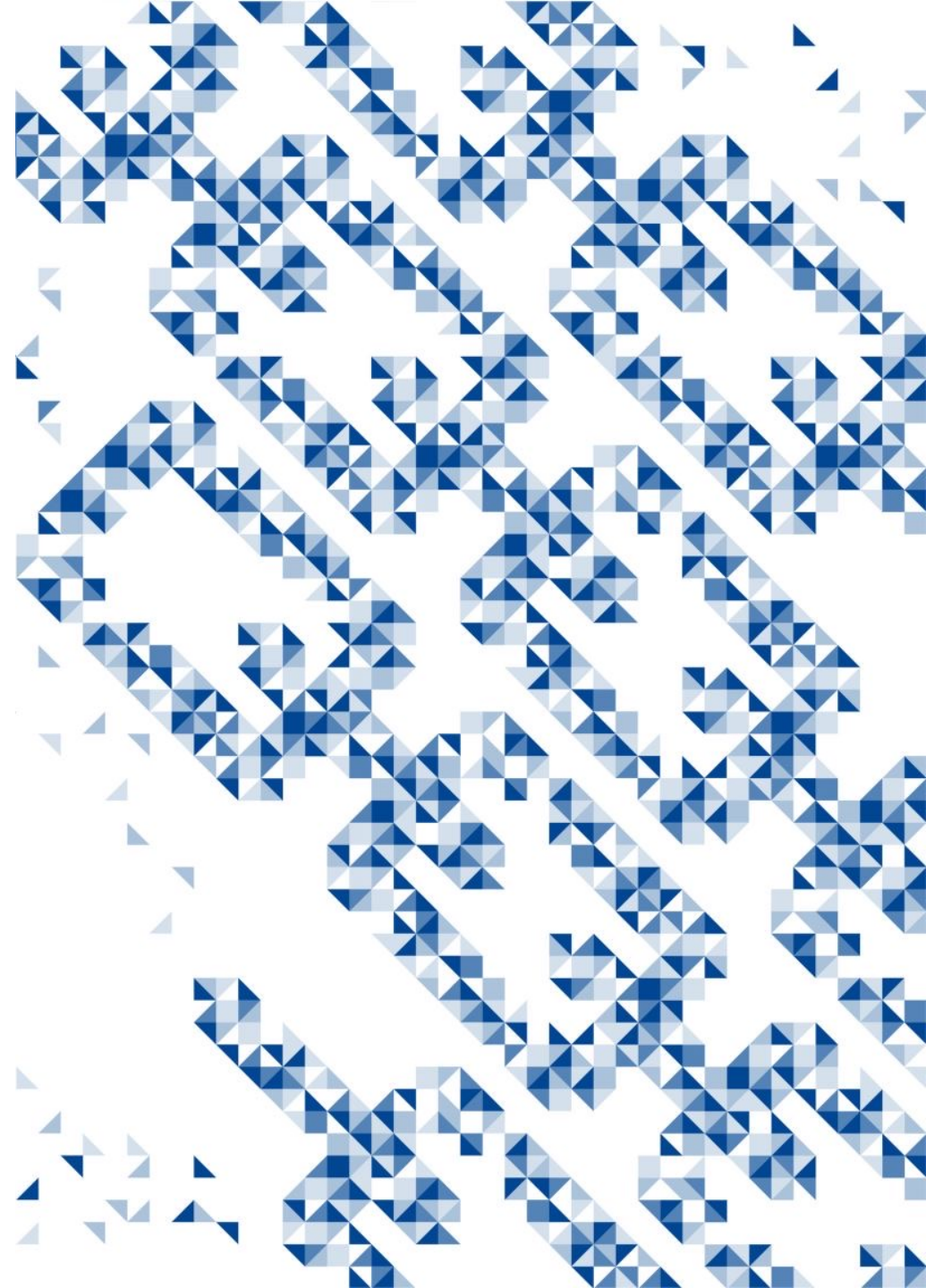
5. Repeat the assignment and update steps until the centroids no longer change.
 - The algorithm aims to minimize the Within-Cluster Sum of Squares (WCSS).
 - WCSS measures the distance between data points and their cluster centroids.



K-Mean algorithm disadvantages

The main disadvantages are:

- K-Means requires the number of clusters (K) to be determined in advance.
- It requires to select the initial centeroids for each cluster.





K-mean clusters(k) number selection

There are two methods to select the optimal number of k:

- Elbow method
- Silhouette method

Elbow method



1. First step:

- Apply the clustering algorithm to the given dataset using different values of K .
- First, set $K=1$ and form the clusters. Then, repeat the process for $K=2$, $K=3$, and so on for several values of K

For example: $k = [1,2,3,4,5,6,7,8,9,10]$

2. Second step :

- Calculate the Within-Cluster Sum of Squares (WCSS). WCSS measures the total distance between data points and their corresponding cluster centroids.
- This calculation is repeated for every value of K .

3. Third step:

- Plot a curve of WCSS , showing the relationship between WCSS and the number of clusters.

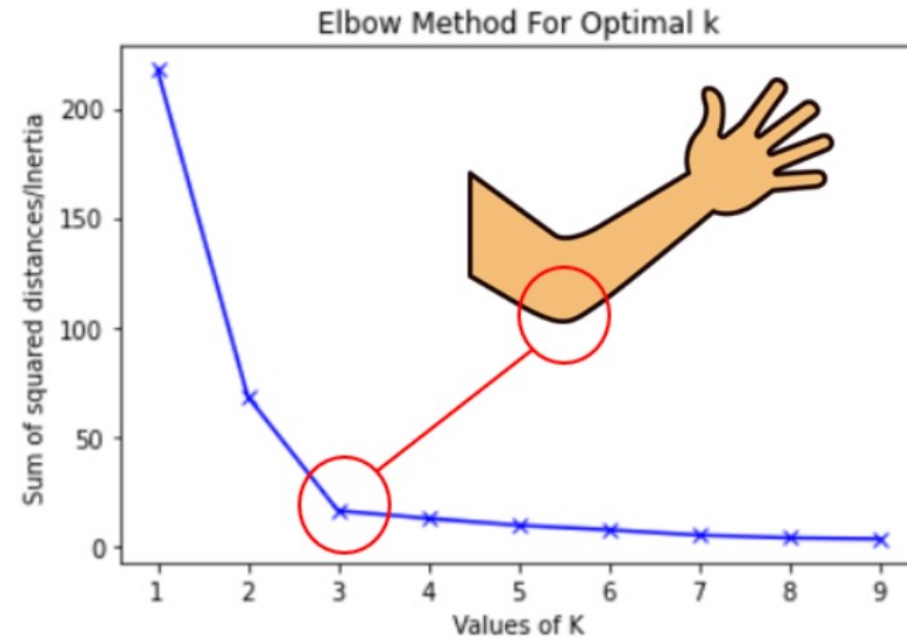
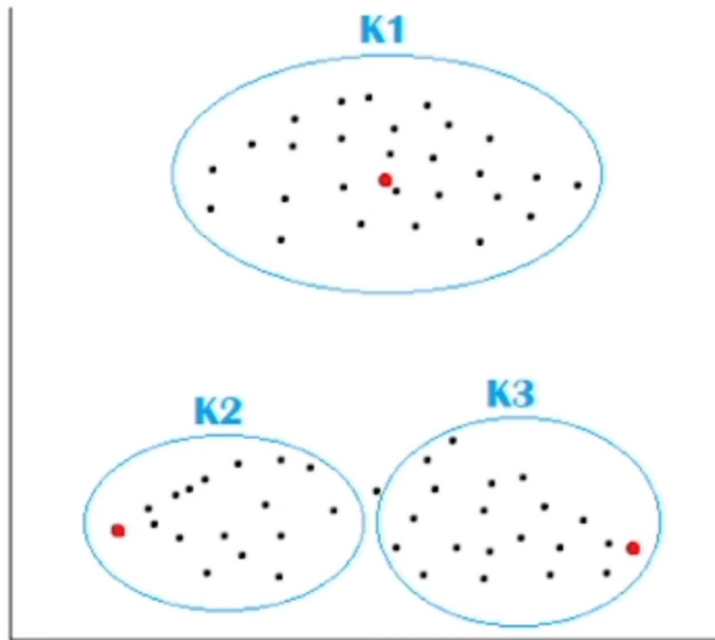
4. Fourth step:

- The location of the bend in the plot is considered an indicator of the approximately number of clusters.

Within-Cluster Sum of Squares (WCSS)

To understand how WCSS is calculated, let us consider the following example:

$$WCSS = \sum_{k=1}^k \sum_{x_i \in C_k} distance(x_i, \mu_k)^2$$



Line plot between K and inertia

Example:

$$WCSS = \sum_{k=1}^k \sum_{x_i \in C_k} distance(x_i, \mu_k)^2$$

- Cluster 1: Points: (1,1),(2,2), Centroid:(1.5,1.5)

WCSS for Cluster 1

$$=((1-1.5)^2+(1-1.5)^2)+((2-1.5)^2+(2-1.5)^2)$$

$$=0.5+0.5=1$$

- Cluster 2: Points: (8,8),(9,9), Centroid:(8.5,8.5)

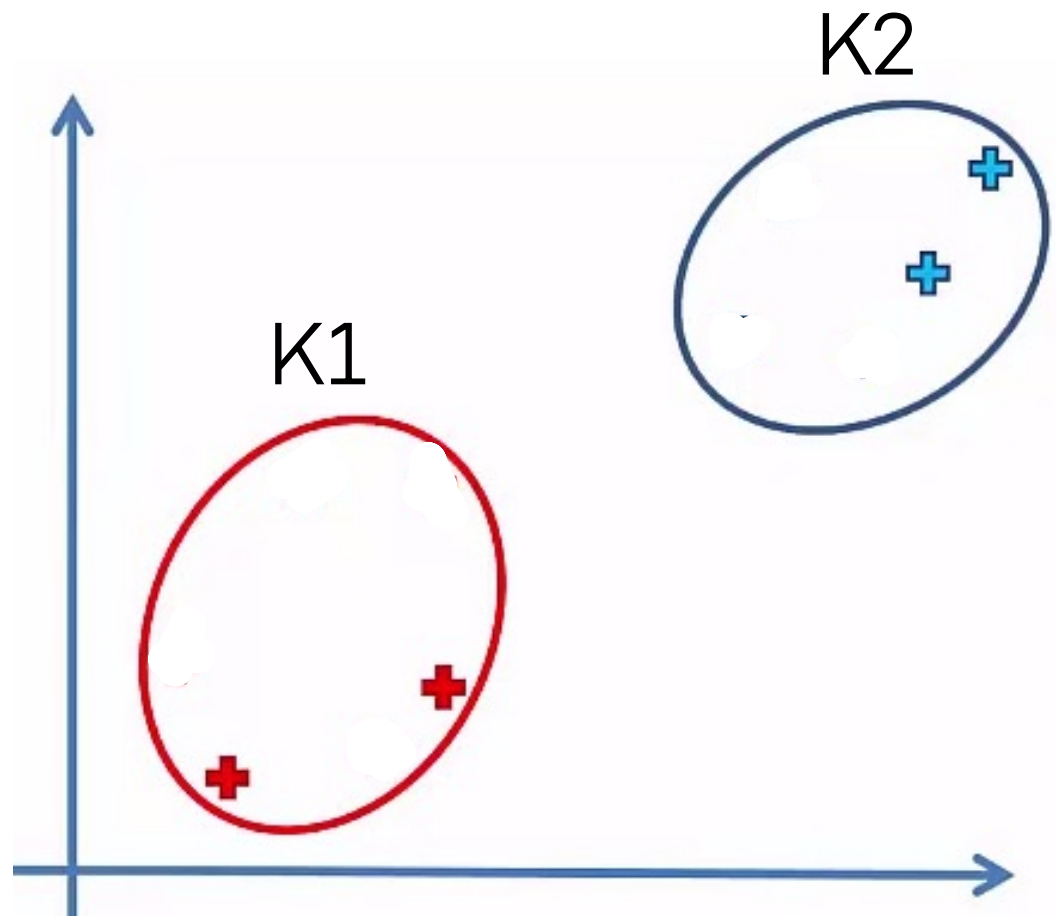
WCSS for Cluster 2

$$=((8-8.5)^2+(8-8.5)^2)+((9-8.5)^2+(9-8.5)^2)$$

$$=0.5+0.5=1$$

Total WCSS

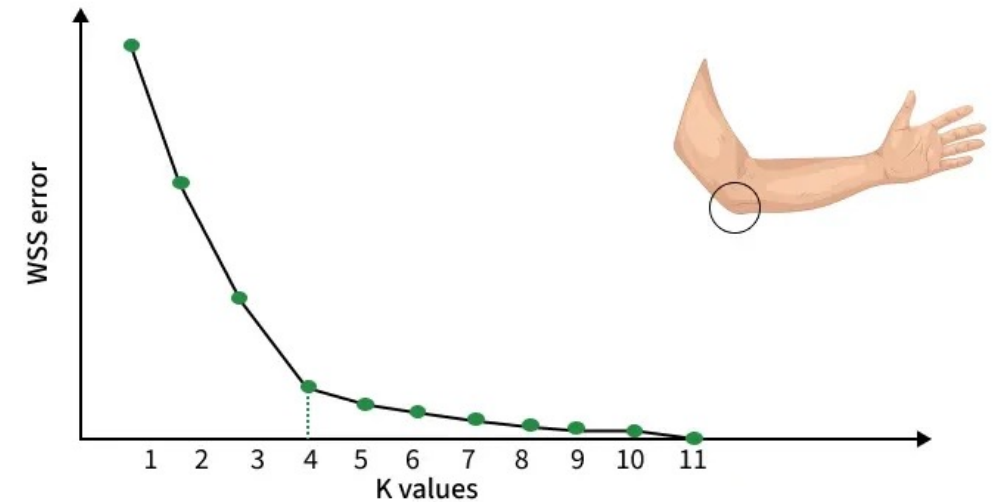
$$WCSS=1+1=2$$



Why Elbow?

1. At first, WCSS decreases rapidly as more clusters are added.
2. After a certain point, the decrease becomes slower.
3. This creates a sharp bend in the curve.
4. That bend resembles an elbow shape:

rapid decrease → bend (elbow) → slow decrease



The elbow point represents the optimal value of K, where adding more clusters no longer provides significant improvement.

Silhouette method

The Silhouette Coefficient or Silhouette Score is a measure used to evaluate the quality of clustering.

It indicates how similar a data point is to other points within its own cluster compared to points in other clusters.

To find the optimal number of clusters, we calculate the Silhouette Coefficient for different values of K , such as $K=2$, $K=3$, $K=4$, and so on.

Then, we plot the Silhouette Score against the number of clusters.

The value of K with the highest score is considered the best, because it indicates compact and well-separated clusters.

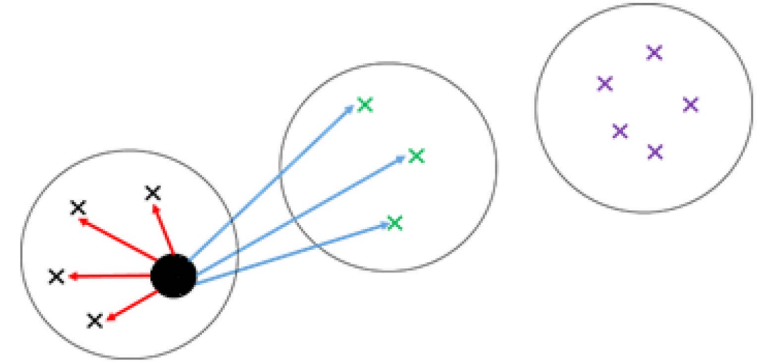
Silhouette coefficient calculation

The Silhouette Score is the average of all data point coefficients.

$$S(i) = \frac{b(i) - a(i)}{\max \{a(i), b(i)\}}$$

Where :

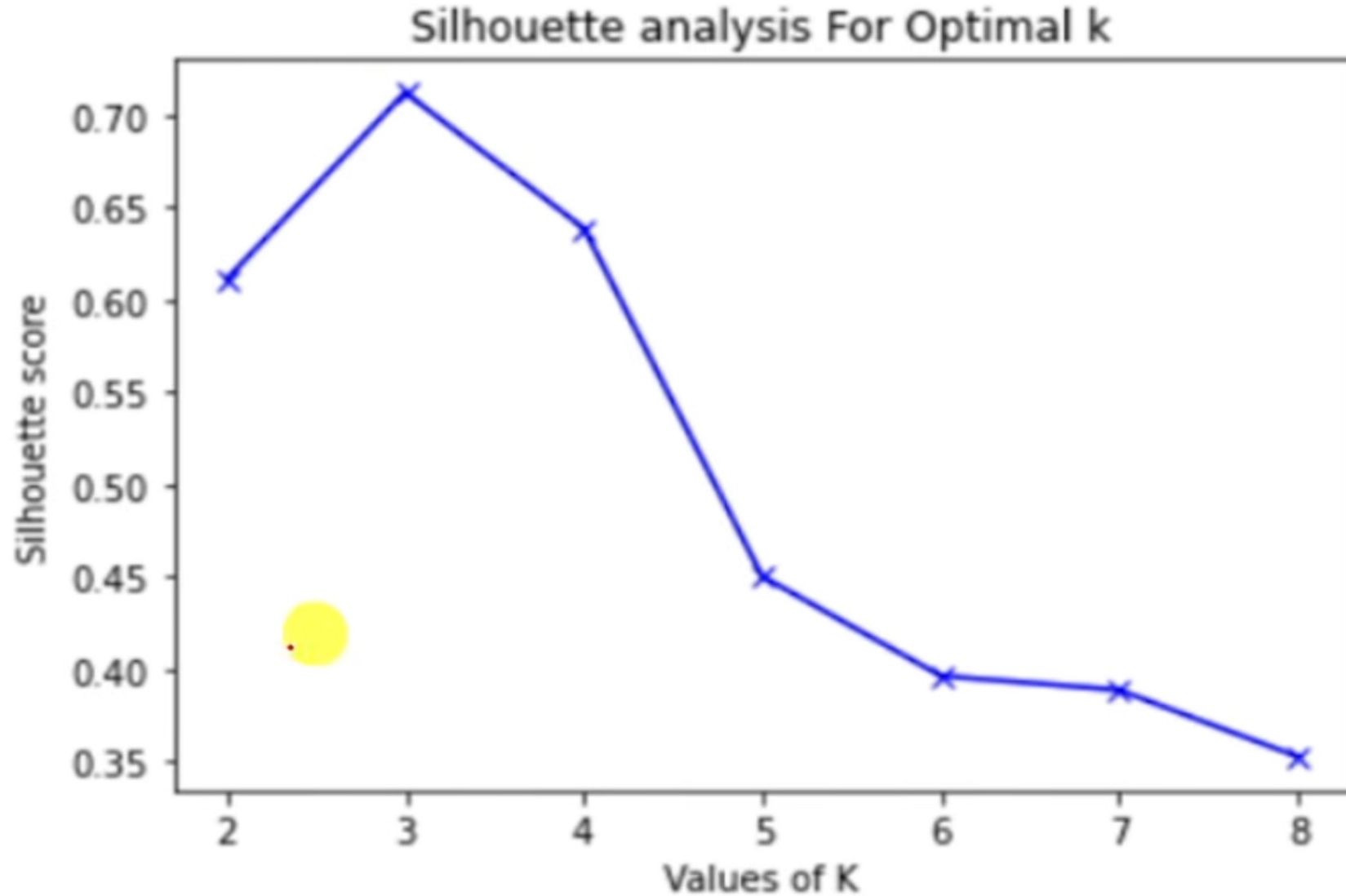
- $s(i)$ = Silhouette Coefficient of data point i
- $a(i)$ = average distance between point i and all other points in the same cluster
- $b(i)$ = average distance between point i and all points in the nearest neighboring cluster



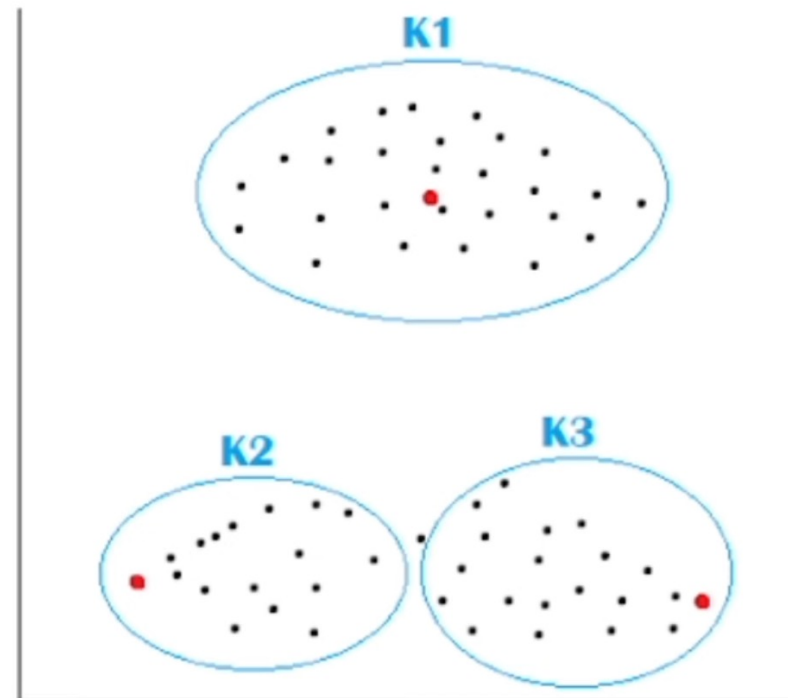
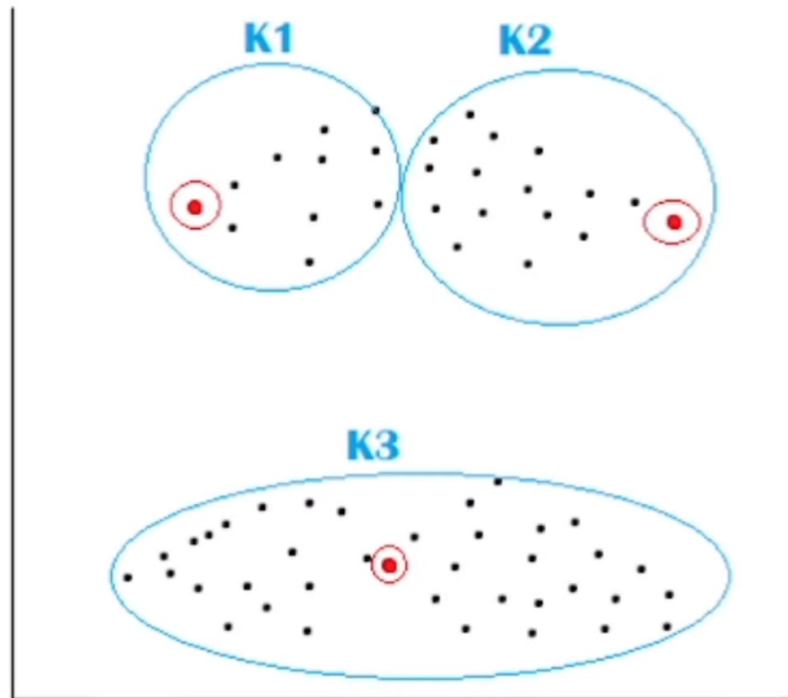
$s(i)$ score interpretation:

- $S \approx 1$ → point is well clustered
- $s \approx 0$. → point lies between clusters
- $S \approx -1$. → point may be assigned to the wrong cluster

$$\text{Silhouette Score} = \frac{\sum S(i)}{N}$$



Select the initial centeroids for each cluster



Steps of K-Means++

1. Select the first centroid randomly from the dataset.
2. Compute the distance between each data point and the nearest centroid.
3. Choose the next centroid from the points that are farthest from the existing centroids.
4. Repeat the process until k centroids are selected.
5. Apply the standard K-Means algorithm using these centroids.

$$P(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{x'} D(\mathbf{x}')^2}$$

Where

- $P(\mathbf{x})$: probability of selecting point $\mathbf{x}$
- $D(\mathbf{x})$: distance from point $\mathbf{x}$ to the nearest existing centroid squared
- $\sum_{x'} D(\mathbf{x}')^2$: Sum of squared distances for all points

Example:

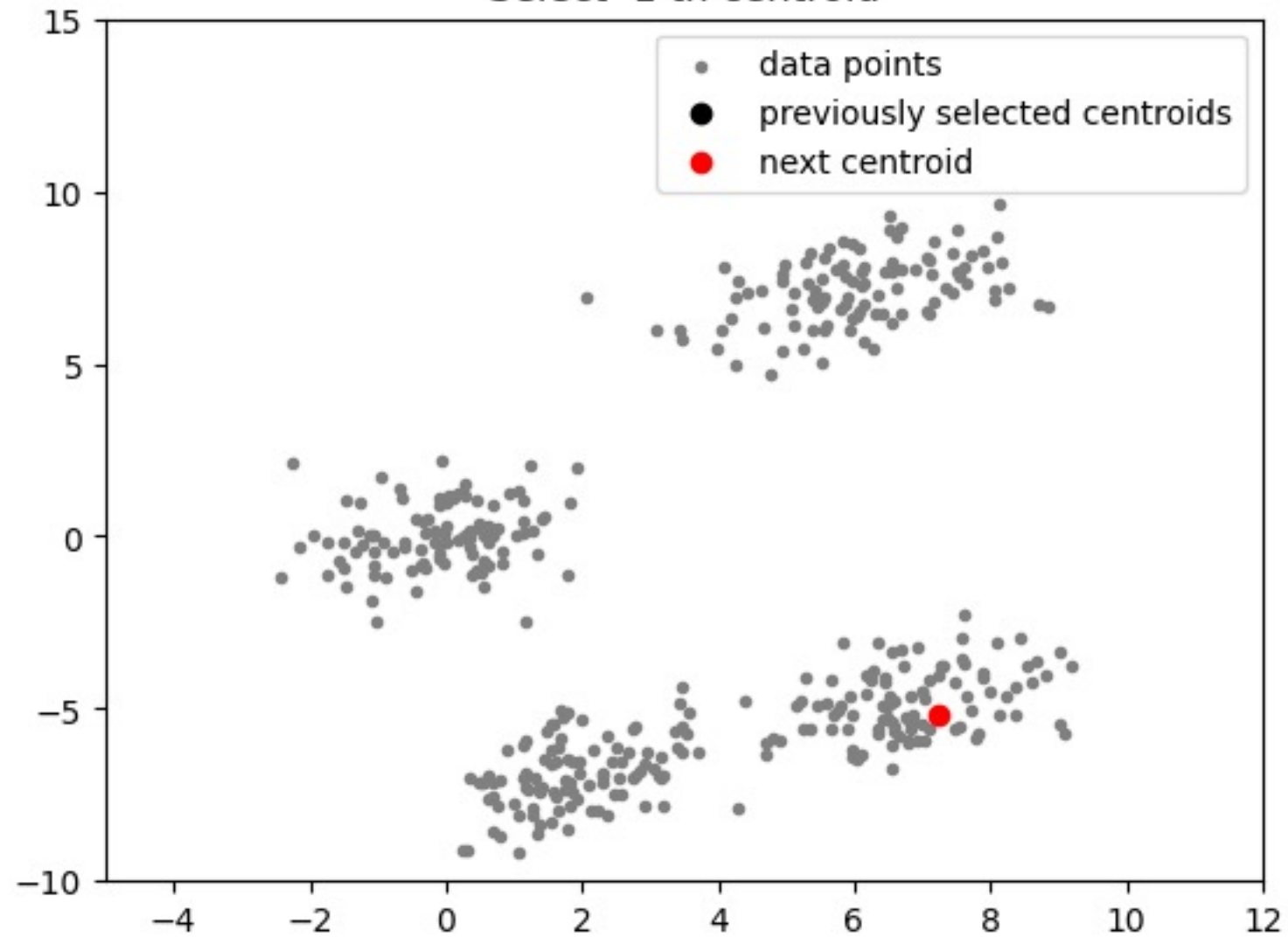
Suppose distances to the nearest centroid are:

Points	Distance $D(x)$	$D(x)^2$
A	1	1
B	2	4
C	3	9

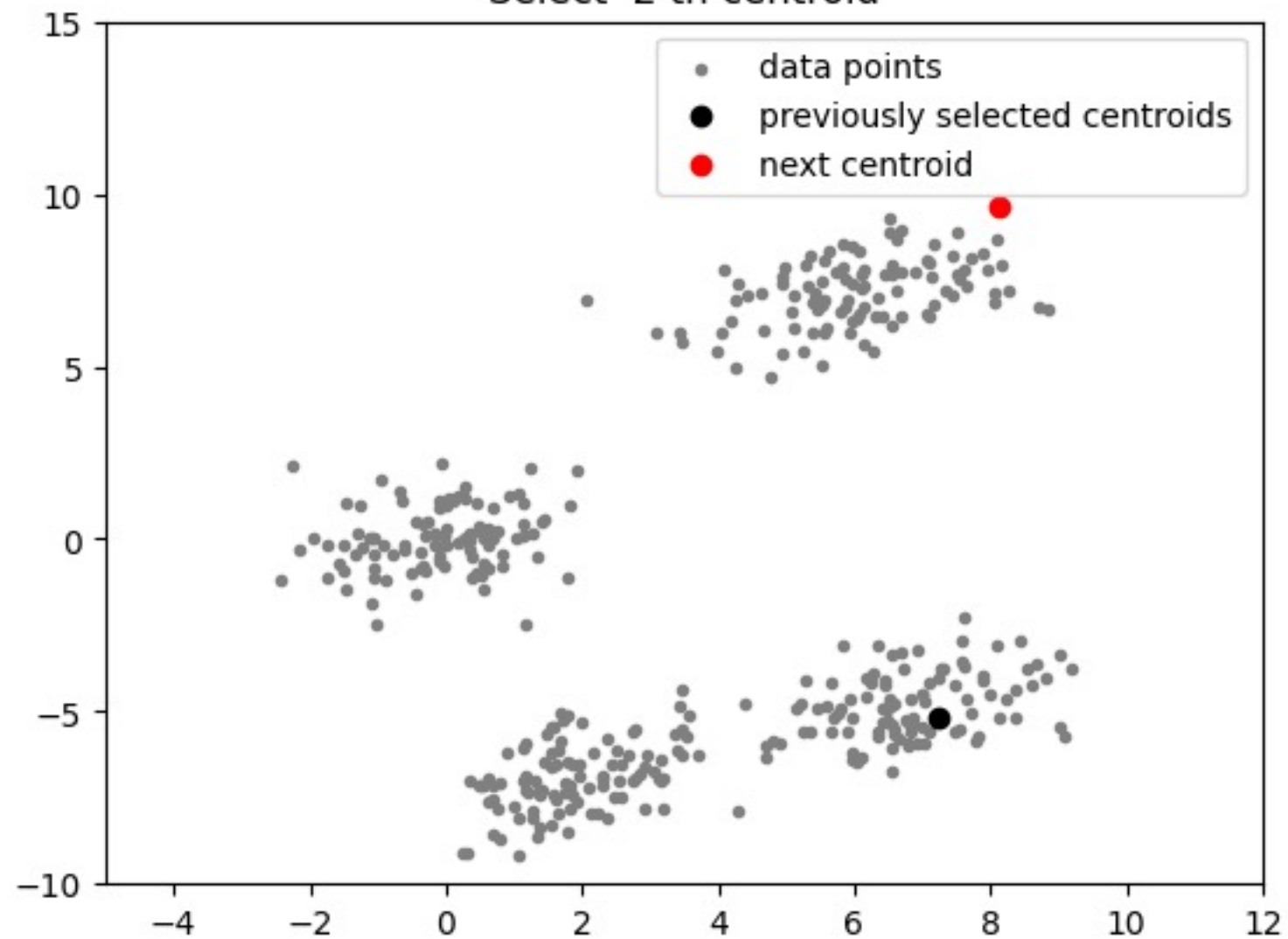
$$P(x) = \frac{D(x)^2}{\sum_{x'} D(x')^2}$$

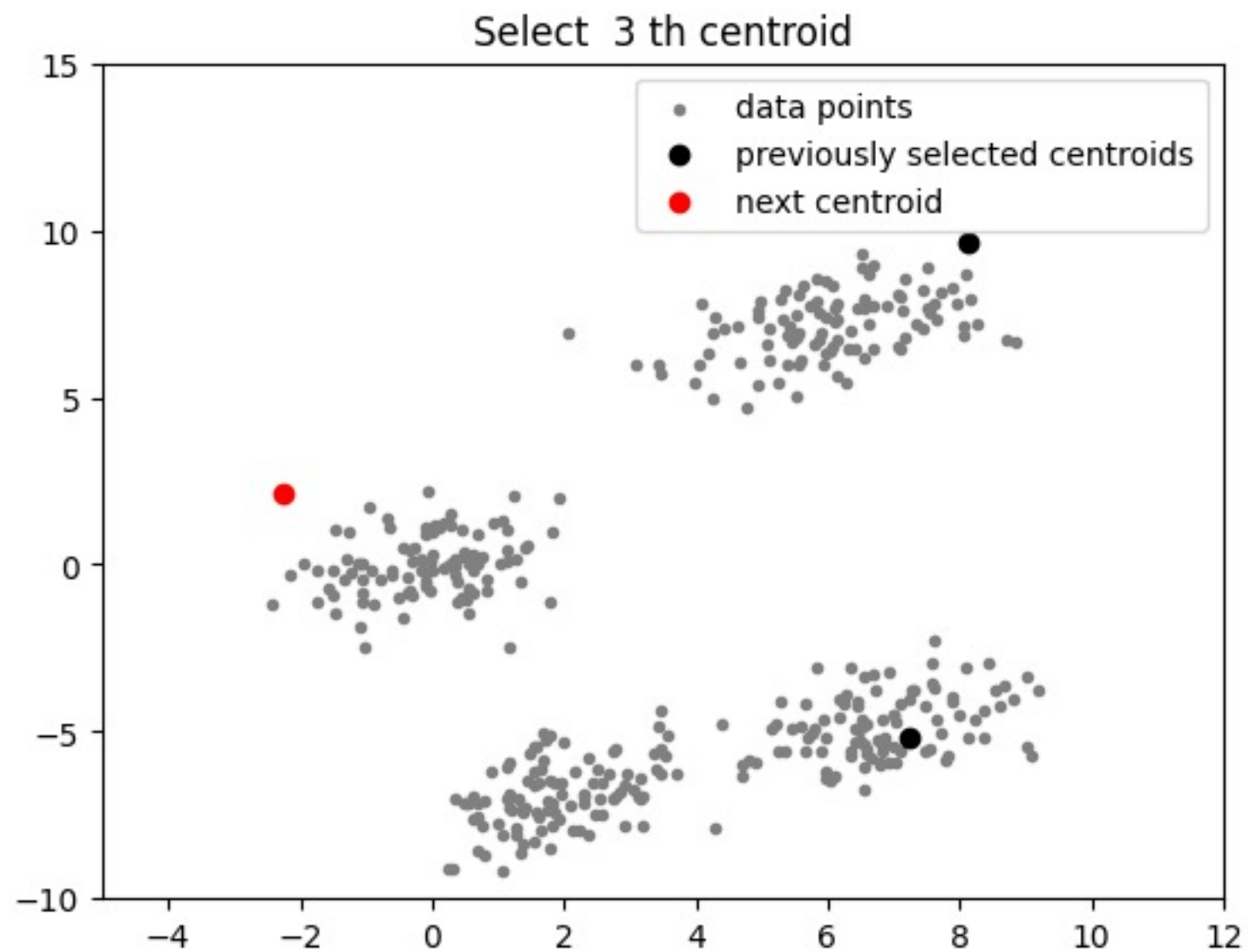
- Sum All Squared Distances = $1+4+9=14$
- Point (A) = $\frac{1}{14} = 0.071$
- Point (B) = $\frac{4}{14} = 0.286$
- Point (C) = $\frac{9}{14} = 0.643$
- Point C has the highest probability because it is farthest from the current centroid.

Select 1 th centroid

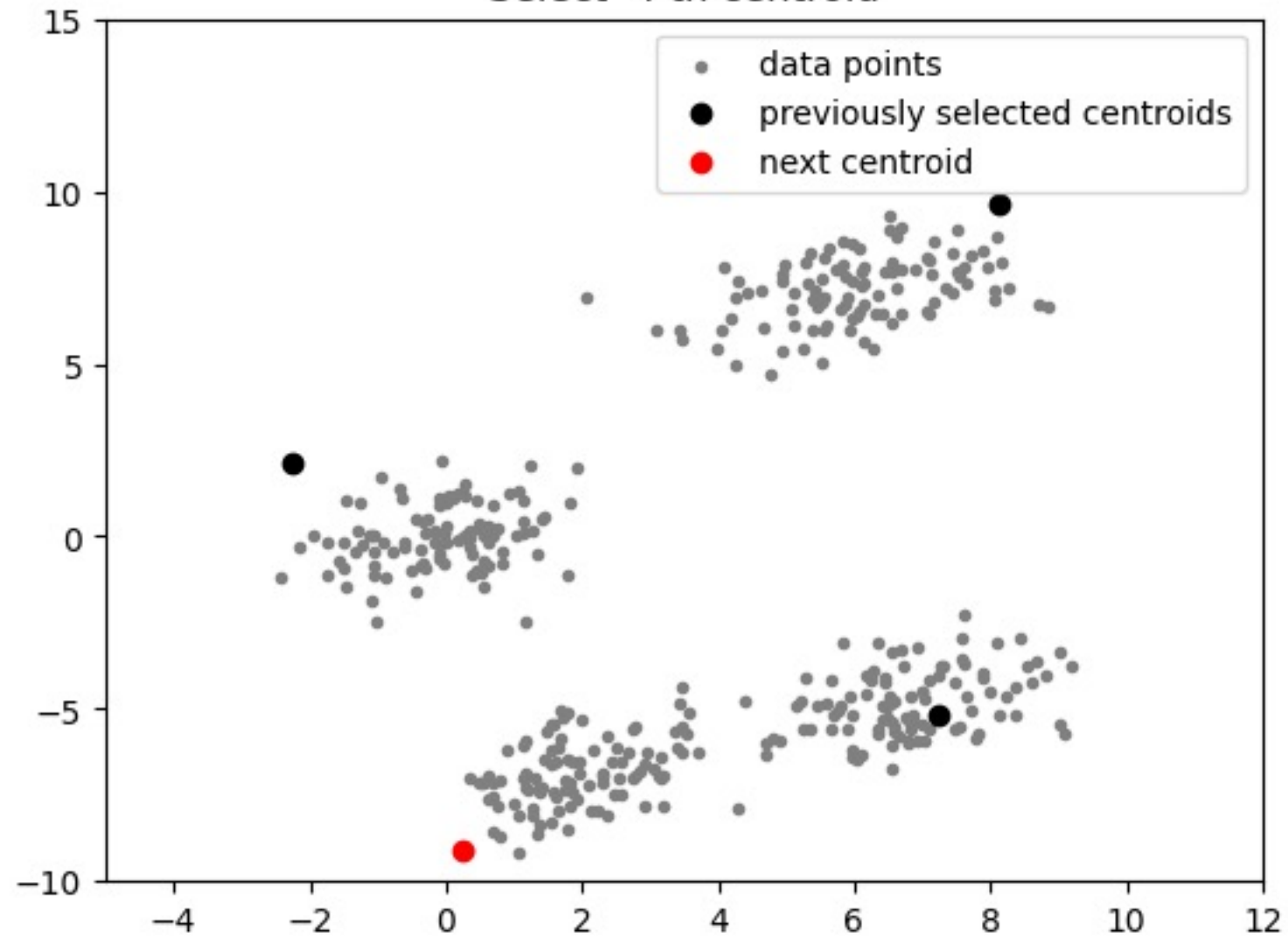


Select 2 th centroid





Select 4 th centroid



Standardization

- Standardization is a feature scaling technique that transforms data by subtracting the mean and dividing by the standard deviation.
- This process makes the features have a mean of 0 and a standard deviation of 1, helping many machine learning algorithms perform more effectively.

$$X_{\text{scaled}} = \frac{X_i - \mu}{\sigma}$$

Where:

- X_i = original value
- μ = mean of the feature
- σ = standard deviation

Characteristics:

- Produces features with mean 0
- Produces features with standard deviation 1
- Works well for approximately normally distributed data
- Commonly used in distance-based algorithms such as K-Means.

K-Mean implementation, Example1

```
#Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Read dataset
df = pd.read_csv("/content/drive/MyDrive/AI course/Employees_income.csv")
df.head()
df.isna().sum()
df.describe()

#Return the number of unique values of all feature
df.nunique()

#Data virtualization
plt.scatter(df["Age"], df["Income ($)"])

#Return columns names
df.columns
```

#Scale the dataset

```
from sklearn.preprocessing import StandardScaler
df_scaled = df
scalar = StandardScaler()
df_scaled[['Age', 'Income($)']] = scalar.fit_transform(df[['Age', 'Income($)']])
df_scaled.head()
```

#Create the K-mean model

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=100)
km.fit(df_scaled[['Age', 'Income($)']])
km.inertia_
```

#Store the labels in the dataframe

```
df_scaled["Label"] = km.labels_
df_scaled.head()
```

#Data virtualization

```
plt.scatter(df_scaled["Age"], df_scaled["Income($)"], c= df_scaled["Label"])
```

K-Mean implementation, Example2

```
#Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Read dataset
df= pd.read_csv("/content/drive/MyDrive/AI course/data_unsupervised_learning.csv")
df.head()
df.isna().sum()

#Return the number of unique values of all feature
df.nunique()

#Eliminate the None values
df.dropna(axis=0, inplace= True)
df.isna().sum()

#Return columns names
df.columns
```



```
#Scale the dataset
```

```
from sklearn.preprocessing import StandardScaler
df_scaled = df
scaler = StandardScaler()
df_scaled[['RHOB', 'GR', 'DEPTH_MD', 'NPHI', 'PEF', 'DTC']] =
scaler.fit_transform(df[['RHOB', 'GR', 'DEPTH_MD', 'NPHI', 'PEF', 'DTC']])
df_scaled.head()
```

```
#Data virtualization
```

```
plt.scatter(df_scaled['RHOB'],df_scaled["NPHI"])
```

```
#Find the Elbow in order to choose the K
```

```
from sklearn.cluster import KMeans
WCSS = []
for k in range(1,10):
    km = KMeans(n_clusters=k, random_state=100)
    km.fit(df_scaled)
    WCSS.append(km.inertia_)
print(WCSS)
plt.plot(range(1,10), WCSS, color="red", marker="o")
```



```
#Create the K-mean model using the optimal K
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, random_state=100)
km.fit(df_scaled)
km.inertia_

#Store the labels in the dataframe
df_scaled["Label"] = km.labels_
df_scaled.head(10)

#Data virtualization
plt.scatter(df_scaled['RHOB'],df_scaled["NPHI"], c=df_scaled["Label"])
```

Association

- Association is a relationship or connection between two or more items, events, or variables.
- Association rules are used to discover relationships and patterns in large datasets.
- In data mining and market basket analysis, association refers to items that frequently occur together in transactions.

$$X \rightarrow Y$$

- Where X and Y are different sets of items.
- The rule means that when X appears, Y is also likely to appear.

For example:

$$\{\text{Bread}, \text{Butter}\} \rightarrow \text{Milk}$$

- Means that customers who buy bread and butter also tend to buy milk.



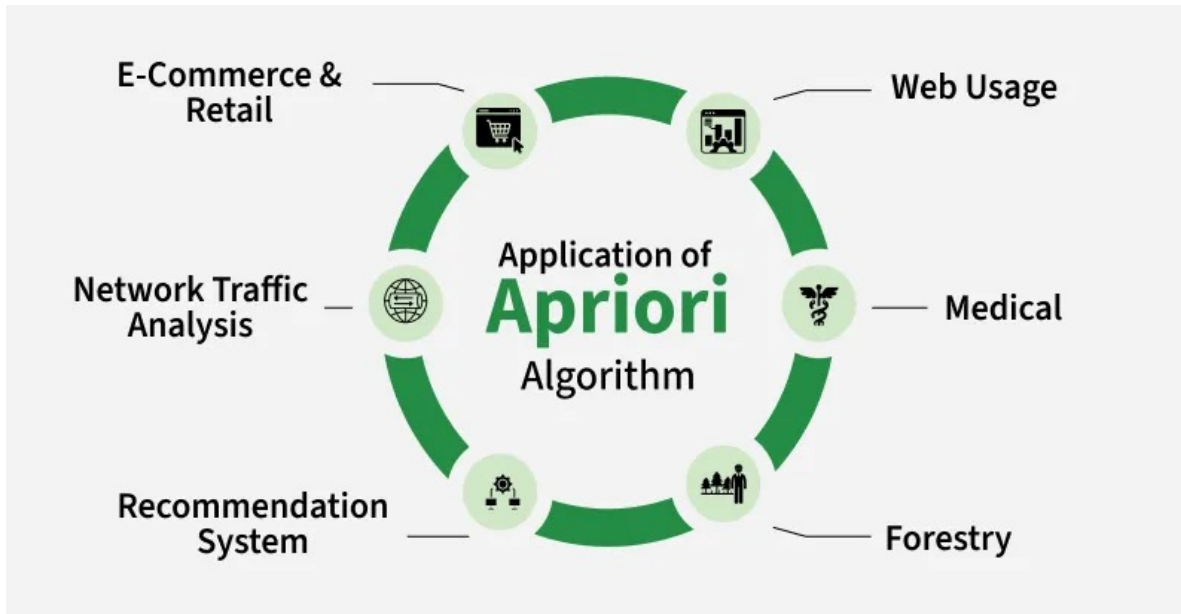
Association rule mining

- Association rule mining is the method used to find these relationships in large datasets.

It involves:

- Finding frequent itemset
 - Calculating support, confidence, and lift
 - Generating strong association rules
-
- Algorithms such as the Apriori algorithm are used for association rule mining.

Apriori algorithm



- Apriori Algorithm is a data mining technique used to identify items that frequently appear together in large datasets.
- It helps discover relationships and association rules between items, making it widely used in market basket analysis.

Apriori algorithm steps

1. Finding frequent itemsets

- The algorithm first counts how often each item appears in the dataset.
- Items that appear more than the minimum support threshold are called frequent itemsets.

2. Generating item groups

- Frequent items are combined to form larger groups, such as pairs, triples, and more.
- The algorithm checks which groups appear frequently in the data.

3. Removing infrequent groups

- If a group does not meet the minimum support, any larger group containing it is ignored.
- This reduces unnecessary calculations and improves speed.

4. Generating association rules

- Finally, the algorithm creates association rules between items and evaluates them using support, confidence, and lift to identify strong relationships.

Key Metrics of Apriori Algorithm

1. Support

- Measures how frequently an item or itemset appears in the dataset relative to the total number of transactions.

$$\text{Support}(X) = \frac{\text{Number of transactions containing } X}{\text{Total number of transactions}}$$

- Indicates the overall occurrence of an itemset.
- Higher support means the itemset appears more frequently.

2. Confidence

- Confidence measures the likelihood that item Y is appeared when item X is appeared.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- Indicates the strength of association between items
- Shows how often items occur together

3. Lift

- Lift measures how much more likely two items are purchased together compared to random chance.

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

Lift Value	Meaning
Lift = 1	No relationship between items
Lift > 1	Positive association (items occur together more than expected)
Lift < 1	Negative association (items occur together less than expected)

- Evaluates the strength of item relationships

Example:

Generate a strong association rule.

Min_support = 40%=0.40

Min_confidence = 70%=0.70

Solution:

Min_support_count = min_support * number of transactions
= 0.40 * 5 = 2

1-itemset	Support_count
Bread	3
Butter	3
Milk	2
Beer	2
Cookies	1
Diapers	3



1-frequent_itemset	Support_count
Bread	3
Butter	3
Milk	2
Beer	2
Diapers	3

Transaction	Items
T1	{Bread, Butter, Milk}
T2	{Bread, Butter}
T3	{Beer, Cookies, Diapers}
T4	{Milk, Diapers, Bread, Butter}
T5	{Beer, Diapers}

{Bread, Butter, Milk, Beer, Diapers}

Transaction	Items
T1	{Bread, Butter, Milk}
T2	{Bread, Butter}
T3	{Beer, Cookies, Diapers}
T4	{Milk, Diapers, Bread, Butter}
T5	{Beer, Diapers}

2-itemset	Support_count
{Bread, Butter}	3
{Bread, Milk}	2
{Bread, Beer}	0
{Bread, Diaper}	1
{Butter, Milk}	2
{Butter, Beer}	0
{Butter, Diapers}	1
{Milk, Beer}	0
{Milk, Diapers}	1
{Beer, Diapers}	2

→

2-frequent_itemset	Support_count
{Bread, Butter}	3
{Bread, Milk}	2
{Butter, Milk}	2
{Beer, Diapers}	2

{Bread, Butter, Milk, Beer, Diapers}

Transaction	Items
T1	{Bread, Butter, Milk}
T2	{Bread, Butter}
T3	{Beer, Cookies, Diapers}
T4	{Milk, Diapers, Bread, Butter}
T5	{Beer, Diapers}

3-itemset	Support_count
{Bread, Butter, Milk}	2
{Bread, Butter, Beer}	0
{Bread, Butter, Diapers}	1
{Bread, Milk, Beer}	0
{Bread, Milk, Diapers}	1
{Bread, Beer, Diapers}	0
{Butter, Milk, Beer}	0
{Butter, Milk, Diapers}	1
{Butter, Beer, Diapers}	0
{Milk, Beer, Diapers}	0

→

3-frequent_itemset	Support_count
{Bread, Butter, Milk}	2

Confidence calculations

1-frequent_itemset	Support_count
Bread	3
Butter	3
Milk	2
Beer	2
Diapers	3

2-frequent_itemset	Support_count
{Bread, Butter}	3
{Bread, Milk}	2
{Butter, Milk}	2
{Beer, Diapers}	2

3-frequent_itemset	Support_count
{Bread, Butter, Milk}	2

Min_confidence = 70%

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Candidate rules are:

- For {Bread, Butter}

$$\text{Confidence}(\text{Bread} \rightarrow \text{Butter}) = \frac{3}{3} = 100\% \text{ Strong rule}$$

$$\text{Confidence}(\text{Butter} \rightarrow \text{Bread}) = \frac{3}{3} = 100\% \text{ Strong rule}$$

- For {Bread, Milk}

$$\text{Confidence}(\text{Bread} \rightarrow \text{Milk}) = \frac{2}{3} = 67\% \text{ Weak rule}$$

$$\text{Confidence}(\text{Milk} \rightarrow \text{Bread}) = \frac{2}{2} = 100\% \text{ Strong rule}$$

- For {Butter, Milk}

$$\text{Confidence}(\text{Butter} \rightarrow \text{Milk}) = \frac{2}{3} = 67\% \text{ Weak rule}$$

$$\text{Confidence}(\text{Milk} \rightarrow \text{Butter}) = \frac{2}{2} = 100\% \text{ Strong rule}$$

Confidence calculations

1-frequent_itemset	Support_count
Bread	3
Butter	3
Milk	2
Beer	2
Diapers	3

2-frequent_itemset	Support_count
{Bread, Butter}	3
{Bread, Milk}	2
{Butter, Milk}	2
{Beer, Diapers}	2

3-frequent_itemset	Support_count
{Bread, Butter, Milk}	2

Candidate rules are:

- For {Beer, Diapers}

$$\text{Confidence}(\text{Beer} \rightarrow \text{Diapers}) = \frac{2}{2} = 100\% \text{ Strong rule}$$

$$\text{Confidence}(\text{Diapers} \rightarrow \text{Beer}) = \frac{2}{3} = 67\% \text{ Weak rule}$$

- For {Bread, Butter, Milk}

$$\text{Confidence}(\text{Bread, Butter} \rightarrow \text{Milk}) = \frac{2}{3} = 67\% \text{ Weak rule}$$

$$\text{Confidence}(\text{Bread, Milk} \rightarrow \text{Butter}) = \frac{2}{2} = 100\% \text{ Strong rule}$$

$$\text{Confidence}(\text{Butter, Milk} \rightarrow \text{Bread}) = \frac{2}{2} = 100\% \text{ Strong rule}$$

$$\text{Confidence}(\text{Bread} \rightarrow \text{Butter, Milk}) = \frac{2}{3} = 67\% \text{ Weak rule}$$

$$\text{Confidence}(\text{Butter} \rightarrow \text{Bread, Milk}) = \frac{2}{3} = 67\% \text{ Weak rule}$$

$$\text{Confidence}(\text{Milk} \rightarrow \text{Bread, Butter}) = \frac{2}{2} = 100\% \text{ Strong rule}$$

Apriori algorithm implementation

```
#Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Read dataset
cols = [f"item{i}" for i in range(1, 21)]
df = pd.read_csv("/content/drive/MyDrive/AI
course/Market_Basket.csv",header=None,names = cols)
df.head()
df.shape
Df.describe()

#Create the transactions
transactions = []
for i in range(0, 7501):
    transactions.append([str(df.values[i,j]) for j in range(0, 20)])

#Print the first five
transactions[0:5]
```

```
#Install apyori model
```

```
!pip install apyori
```

```
#Create the association rule
```

```
from apyori import apriori
```

```
rules= apriori(transactions,min_support =0.003, min_confidence = 0.2, min_lift = 3,  
min_length = 2)
```

```
#Print the rules
```

```
result = list(rules)
```

```
for i in result:
```

```
print(i)
```